

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 02 -

INTRODUÇÃO AO DOCKER

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	12
O QUE VOCÊ VIU NESTA AULA?	13
REFERÊNCIAS.....	14

EMSE

O QUE VEM POR AÍ?

Depois de compreender os fundamentos de virtualização e containers, chegou a hora de colocar a mão na massa com a principal tecnologia que popularizou esse modelo no mercado: o Docker. Nesta aula, você vai entender como o Docker organiza o ciclo de construção, distribuição e execução de aplicações e por que ele se tornou peça central em projetos de Machine Learning que precisam sair do notebook e ganhar escala com previsibilidade.

Ao longo do conteúdo, você aprenderá a ler a arquitetura do Docker, criar imagens para aplicações de ML, executar e gerenciar containers localmente e entender como volumes, variáveis de ambiente e logs afetam a operação real de um serviço de inferência. O foco é transformar conceito em capacidade prática para reduzir falhas de ambiente e acelerar entregas.

HANDS ON

No hands on desta aula, a proposta é compreender uma aplicação simples de inferência em Docker, observando cada etapa com olhar de engenharia: build da imagem, execução do container, injeção de variáveis de ambiente e coleta de logs. A videoaula mostrará não só os comandos, mas o raciocínio por trás de cada decisão para que você consiga repetir o processo em cenários reais.

A ideia é levar os conceitos fundamentais da utilização de container e se aprofundar na construção de imagens e empacotamento das suas aplicações. Vamos entender o ecossistema do Docker e as camadas declarativas de um Dockerfile e, além disso, vamos começar a executar nossos primeiros containers.

Para acompanhar, use os comandos de apoio:

- ``docker version``,
- ``docker info``,
- ``docker build -t ml-inferencia:1.0.0 .``,
- ``docker run --rm -p 8000:8000 ml-inferencia:1.0.0``,
- ``docker ps``, ``docker logs <container_id>``,
- ``docker exec -it <container_id> bash``,

SAIBA MAIS

Introdução prática: por que Docker é central em Engenharia de ML

Anteriormente, você viu que containers resolvem boa parte do problema de portabilidade. Nesta aula, o Docker aparece como ferramenta concreta para transformar esse princípio em rotina operacional. Em termos simples, o Docker permite empacotar aplicação, dependências e configurações em uma imagem versionada, que pode ser executada com o mesmo comportamento em ambientes diferentes.

Em projetos de Machine Learning, isso evita divergências entre quem treina, quem valida e quem opera o modelo em produção. O ganho não está apenas na execução técnica, mas na colaboração entre times. Quando todos trabalham com a mesma imagem, o debate deixa de ser “funciona no meu ambiente” e passa a ser “qual versão devemos promover”.

Arquitetura do docker: client, daemon e registry

A arquitetura do Docker pode ser entendida por três blocos principais:

- Docker Client: interface usada para executar comandos (``docker build``, ``docker run``, ``docker push``).
- Docker Daemon: serviço que realmente constrói imagens, cria containers e gerencia recursos locais.
- Docker Registry: repositório de imagens, público ou privado, usado para compartilhar e versionar artefatos.

Quando você roda ``docker build``, o Client envia instruções para o Daemon. O Daemon processa o Dockerfile, monta as camadas e produz uma imagem. Se necessário, ele busca imagens base em um Registry. Esse fluxo parece simples, mas representa uma cadeia de entrega completa para aplicações de ML.

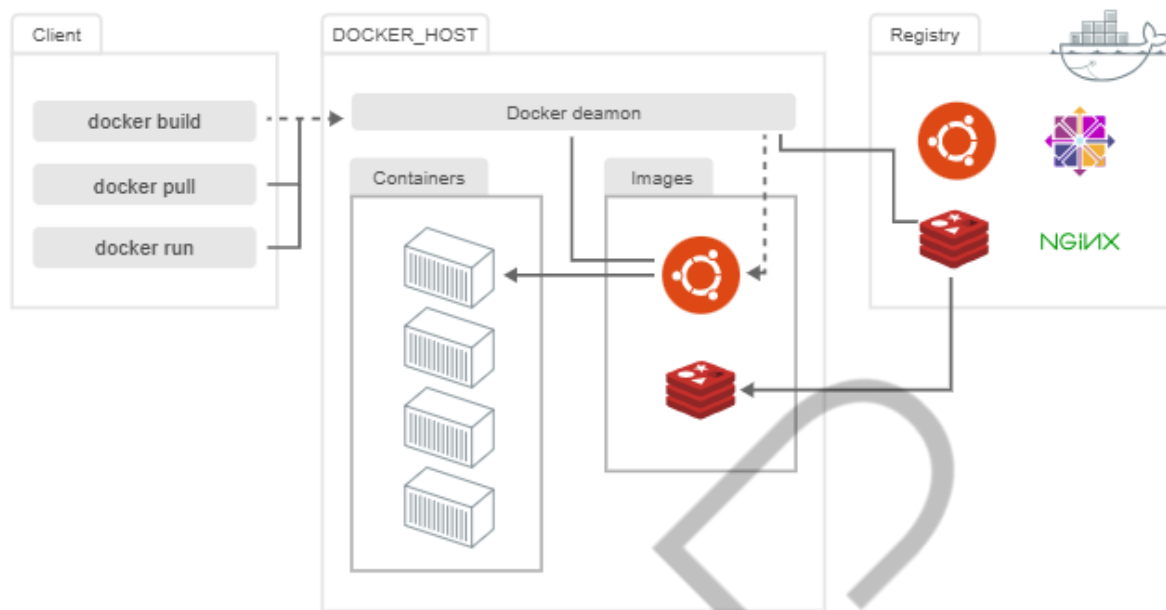


Figura 1 - Relação entre Docker Client, Docker Daemon e Docker Registry
Fonte: Cunha (2019)

Imagens, containers e Dockerfile: diferença que muda sua operação

Imagem e container não são sinônimos. A imagem é o artefato imutável de build; o container é a instância em execução dessa imagem. Em termos didáticos:

- Imagem: “pacote” versionado.
- Container: “processo” rodando a partir do pacote.

O Dockerfile é o arquivo declarativo que define como a imagem será construída. Instruções como ``FROM``, ``WORKDIR``, ``COPY``, ``RUN``, ``EXPOSE``, ``CMD`` e ``ENTRYPOINT`` formam o roteiro de build. Em ML, esse roteiro precisa ser pensado para reprodutibilidade: escolher imagem base adequada, instalar dependências com versões fixas e separar camadas estáveis das variáveis.

Exemplo de leitura arquitetural do Dockerfile:

- ``FROM python:3.11-slim`` define o ambiente base.
- ``WORKDIR /app`` organiza contexto de execução.
- ``COPY requirements.txt .`` + ``RUN pip install ...`` cria camada de dependências.

- ``COPY . .`` adiciona código da aplicação.
- ``CMD ["python", "app.py"]`` define processo principal.

Essa ordem afeta diretamente o cache de build. Se você copia todo o código antes de instalar dependências, perde eficiência de cache e torna builds mais lentos sem necessidade.

```
# =====  
# Stage 1 - Build  
# =====  
FROM node:20.11.1-alpine3.18 AS build  
  
# Definir diretório de trabalho  
WORKDIR /app  
  
# Copiar apenas arquivos necessários para instalar dependências  
COPY package.json package-lock.json ./  
  
# Instalar dependências de forma determinística  
RUN npm ci --only=production  
  
# Copiar código da aplicação  
COPY src ./src
```

Figura 2 - Como Dockerfile gera imagem e imagem gera container em runtime
Fonte: Elaborado pelo autor (2026)

Construindo imagens para aplicações de ML

Uma aplicação de inferência geralmente possui três componentes mínimos: carregamento de modelo, endpoint de predição e tratamento de entrada/saída. Ao criar imagem para esse tipo de aplicação, os pontos de atenção mais importantes são:

- Consistência de versões de bibliotecas.
- Tempo de inicialização do container.
- Tamanho final da imagem.
- Previsibilidade de execução.

O uso de ``requirements.txt`` fixado, combinado com uma imagem base enxuta, tende a melhorar performance operacional. Além disso, em aplicações de ML, dependências nativas podem impactar compatibilidade de CPU e sistema operacional. Por isso, sempre valide o comportamento da imagem no ambiente mais próximo possível da produção.

Também é recomendável manter o contexto de build limpo usando ``dockerignore``. Arquivos de notebook, dados temporários e artefatos de treino não devem entrar na imagem de inferência sem necessidade. Essa decisão reduz superfície de ataque, melhora tempo de transferência e simplifica troubleshooting.

Executando e gerenciando containers localmente

O comando ``docker run`` faz mais do que “iniciar um container”: ele também pode definir mapeamento de portas, variáveis de ambiente, volumes e políticas de reinício. Para ML, um cenário comum é expor uma API local para testes:

```
`docker run --rm -p 8000:8000 --name inferencia-api ml-inferencia:1.0.0`
```

Com isso, você consegue testar chamadas HTTP no host enquanto a aplicação roda isolada. Em seguida, comandos como ``docker ps``, ``docker logs`` e ``docker exec`` permitem observar estado, logs e shell interno do container, etapa essencial para depuração.

Outro ponto importante é o ciclo de vida. Containers são descartáveis por natureza. Você deve assumir que podem ser recriados a qualquer momento. Essa mentalidade melhora desenho de aplicações stateless e prepara terreno para orquestração com Kubernetes nas próximas aulas.

Volumes: persistência e separação de responsabilidades

Sem volume, dados gravados dentro do filesystem do container tendem a ser efêmeros. Isso pode ser aceitável para cache temporário, mas perigoso para qualquer informação que precise sobreviver ao ciclo de vida da instância.

Volumes Docker permitem persistir dados fora da camada efêmera do container. Em ML, isso aparece em cenários como:

- Armazenamento de logs de inferência para auditoria.
- Cache de modelos baixados em tempo de execução.
- Compartilhamento de artefatos entre processos locais.

Exemplo:

```
`docker run --rm -v ml_logs:/app/logs ml-inferencia:1.0.0`
```

Essa separação entre aplicação e dados facilita manutenção, backup e observabilidade.

```
# Stage 2 - Runtime
# =====
FROM node:20.11.1-alpine3.18

# Criar usuário não-root
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

# Definir diretório
WORKDIR /app

# Copiar apenas artefatos necessários do estágio de build
COPY --from=build /app /app

# Ajustar permissões
RUN chown -R appuser:appgroup /app

# Definir variáveis de ambiente
ENV NODE_ENV=production \
    PORT=3000

# Trocar para usuário não-root
USER appuser

# Expor porta (documentacional)
EXPOSE 3000

# Comando de inicialização
CMD ["node", "src/index.js"]
```

Figura 3 - Relação entre persistência e configuração
Fonte: Elaborado pelo autor (2026)

Variáveis de ambiente: configuração sem acoplamento ao código

Configuração hardcoded em código é uma das principais fontes de fragilidade operacional. O Docker facilita o desacoplamento por meio de variáveis de ambiente, permitindo mudar comportamento sem reconstruir imagem.

Exemplo:

```
`docker run --rm -e MODEL_VERSION=v2 -e LOG_LEVEL=INFO ml-inferencia:1.0.0`
```

Com isso, a mesma imagem pode atender diferentes contextos (desenvolvimento, homologação e produção) apenas alterando parâmetros de execução. Essa prática é fundamental para promover artefatos com segurança e rastreabilidade.

Logs: da depuração local à observabilidade de produção

Logs são o primeiro instrumento de diagnóstico em incidentes de inferência. No Docker, o comando ``docker logs`` permite acesso rápido à saída padrão do processo principal. Para ser útil, o log precisa ser estruturado e orientado à decisão, incluindo timestamp, nível, contexto de requisição e mensagens acionáveis.

Exemplo de informação útil em log de inferência:

- Versão do modelo em uso.
- Tempo de processamento.
- Status da predição.
- Detalhes de erro sem expor dados sensíveis.

Mesmo em ambiente local, acostumar-se a observar logs de forma disciplinada reduz tempo de detecção de problemas e prepara o time para padrões de observabilidade em Kubernetes.

Erros comuns ao iniciar com Docker em ML

Erro comum 1: construir imagens gigantes com arquivos desnecessários.

Correção: usar `.dockerignore`, imagens base enxutas e camadas bem organizadas.

Erro comum 2: não fixar versão de dependências.

Correção: travar versões e revisar mudanças de bibliotecas antes de promover.

Erro comum 3: misturar configuração de ambiente no código.

Correção: parametrizar execução com variáveis de ambiente.

Erro comum 4: tratar container como máquina virtual.

Correção: adotar mentalidade de processo descartável e stateless sempre que possível.

Erro comum 5: ignorar logs e telemetria no início do projeto.

Correção: padronizar logs desde o primeiro protótipo de inferência.

Com a base do Docker consolidada, o próximo passo é elevar maturidade técnica com boas práticas específicas para ML: otimização de imagem, segurança, versionamento semântico e estratégias de build mais eficientes. A próxima aula vai mostrar como sair do “funciona” para o “funciona com qualidade de produção”.

MERCADO, CASES E TENDÊNCIAS

O mercado de tecnologia já consolidou o Docker como padrão de empacotamento de aplicações modernas e, no contexto de Machine Learning, esse movimento é ainda mais evidente. Empresas que operam múltiplos modelos em produção perceberam que a dificuldade não estava apenas em treinar modelos com boa acurácia, mas em garantir que o mesmo artefato funcionasse com consistência do ambiente de desenvolvimento até a produção. A adoção de imagens versionadas e pipelines de build padronizados reduziu falhas de deploy e acelerou o tempo de entrega de melhorias.

Em casos corporativos, uma prática recorrente é a criação de golden images internas com bibliotecas homologadas, políticas de segurança e padrões de logging já incorporados. Isso diminui variabilidade entre equipes e reduz risco operacional. Também cresce a integração de scanners de vulnerabilidade no ciclo de build, reforçando que segurança em containers precisa começar no desenvolvimento e não apenas no ambiente final.

Outra tendência forte é o uso de plataformas self-service para times de dados, nas quais cientistas e engenheiros conseguem empacotar e publicar serviços de inferência com templates prontos de Dockerfile, observabilidade e esteiras de CI/CD. Esse modelo reduz dependência de operações para tarefas repetitivas e melhora governança. Para profissionais de pós-graduação que atuarão em projetos de ML de ponta a ponta, dominar o Docker com visão de produto, operação e segurança já é competência de base para o mercado.

O QUE VOCÊ VIU NESTA AULA?

Você aprendeu como o Docker funciona por dentro e por que ele é essencial para transformar aplicações de ML em artefatos portáteis e executáveis com consistência. Entendeu a arquitetura Client/Daemon/Registry, diferenciou imagem de container, praticou conceitos de Dockerfile e viu como volumes, variáveis de ambiente e logs influenciam a operação real. Com essa base, você está preparado(a) para avançar para otimizações e boas práticas de qualidade na próxima aula.

REFERÊNCIAS

BURNS, B.; BEDA, J.; HIGHTOWER, K. **Kubernetes**: Up and Running. Sebastopol: O'Reilly Media, 2022.

CUNHA, C. R. **Servidores para quem tem preguiça** — Docker. 2019. Disponível em: <https://medium.com/cristiano-cunha/servidores-para-quem-tem-preguica-docker-5b2e9f760b34>. Acesso em: 06 abr. 2026.

MERKEL, D. **Docker**: Lightweight Linux Containers for Consistent Development and Deployment. Houston: Belltown Media, 2014.

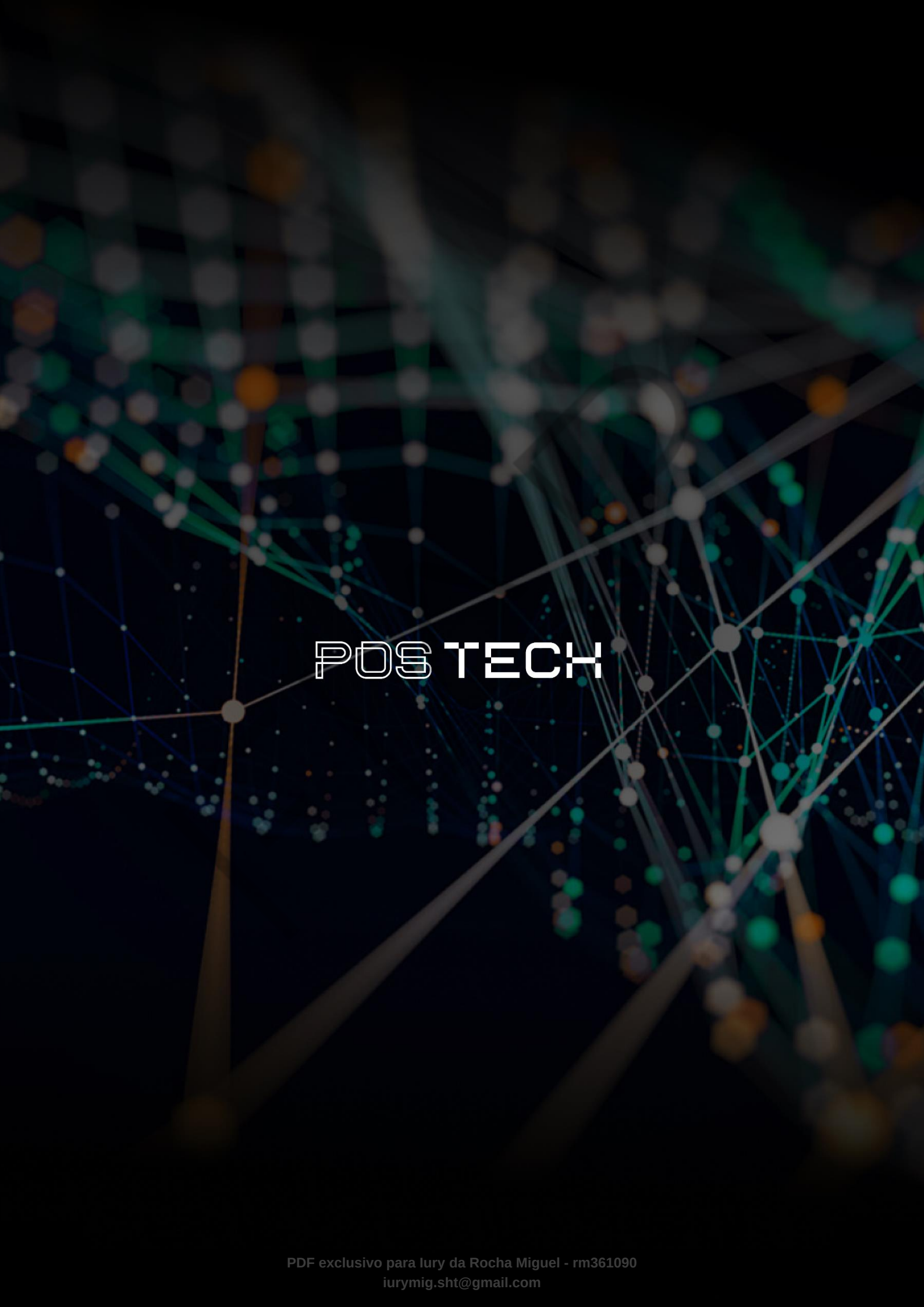
SATO, D. **DevOps na prática**: Entrega de software confiável e automatizada. São Paulo: Casa do Código, 2014.

VERAS, M. **Virtualização**: Da teoria a soluções práticas. São Paulo: Novatec, 2019.

PALAVRAS-CHAVE

DevOps. Containers. Docker. Kubernetes.

EMSE



POSTECH